从头讲解vLLM推理加速原理

Original 程工 程工的文件夹 2024年08月24日 23:35

大型语言模型（LLM）在自然语言处理领域展现出了强大的能力，但同时也带来了巨大的计算挑战，尤其是在推理阶段。这些模型通常包含数十亿甚至数千亿个参数，在生成文本时需要进行大量的计算操作。为了提高推理效率，研究人员开发了各种优化技术，其中vLLM推理加速技术是一种效果显著的方法。vLLM通过综合运用多种优化策略，显著提高了大型语言模型的推理速度和资源利用率。本文将详细探讨vLLM推理加速的核心原理，包括自回归模型的基本概念、注意力机制中的Q、K、V，以及关键的KV缓存技术，并深入分析vLLM所采用的各种优化策略。

一. 自回归模型基础

自回归模型是大型语言模型的基础架构之一。这类模型的核心特征是逐个生成序列中的元素，每个新元素的生成都依赖于之前已生成的所有元素。在语言模型中，这些元素通常是词元（tokens）。

自回归过程可以描述如下:

1) 模型接收初始输入序列。

2) 基于当前序列，预测下一个最可能的词元。

3) 将预测的词元添加到序列末尾。

4) 重复步骤2和3，直到达到预定的终止条件（如生成特定的结束标记或达到最大长度）。

这个过程虽然直观，但在处理长序列时会面临严重的性能瓶颈。因为对于序列中的每个新位置，模型都需要处理从开始到当前位置的所有信息。这就引出了优化的需求，而理解注意力机制是优化的关键。

二、 注意力机制

注意力机制，特别是自注意力（Self-Attention）机制，是现代大型语言模型的核心组成部分。它允许模型在处理序列数据时，动态地关注不同部分的信息。注意力机制的核心概念是Query（查询）、Key（键）和Value（值）。

- Query (Q)：当前单词的一种表示，用于对所有其他单词进行评分（使用它们的Key）。我们只关心当前正在处理的token的Query 。在生成任务中，通常是最后一个token的表示。

- Key (K)：是该句段中所有单词的标签。它们是我们在搜索相关单词时所匹配的内容。用于与Query进行匹配，决定应该关注哪些信息。

- Value (V)：是实际的单词表示，一旦我们对每个单词的相关性进行了评分，这些就是我们聚合起来用来表示当前单词的值。

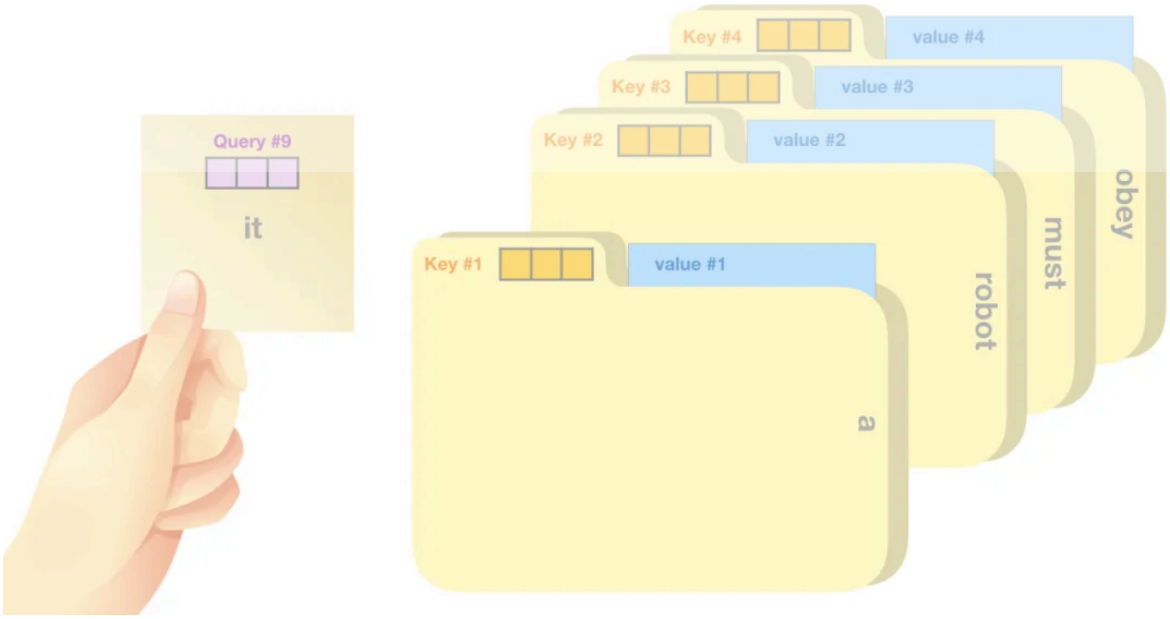
1.工作原理类比：

假设输入为“A robot must obey the orders given it by human beings except where such orders would conflict with the First Law.”

Query (查询)：你手里拿着一张便利贴,上面写着"it"。这就像是当前词的查询向量。在我们的例子中,这是第9个位置的查询向量。

Key (键)：文件柜中每个文件夹的标签就像是键向量。它们代表了序列中每个词的"标识"。在图中,我们看到了4个文件夹,分别标记为"a", "robot", "must", "obey"。

Value (值)：每个文件夹里的实际内容对应着值向量,包含了词的具体信息。



2.注意力计算过程:

你拿着写有"it"的便利贴(Query #9),与每个文件夹的标签(Key #1到#4)进行比较。

比较的结果决定了你对每个文件夹内容的关注程度。

在第二张图中,我们可以看到比较的结果:

"a"文件夹得到30%的关注

"robot"文件夹得到50%的关注

"must"和"obey"文件夹各得到0.1%的关注

这些百分比实际上就是注意力权重,决定了你从每个文件夹中提取信息的比例。



信息综合:

最后,你会根据这些权重从各个文件夹中提取信息,并将它们综合起来,形成对"it"这个词的理解。

最终的表示是多个信息源的加权组合。这个加权组合可以用一个简单的形式表达:

it的表示 = 0.3a + 0.5robot + 0.001must + 0.001obey

在这个表达式中，每个词前的系数代表了注意力权重，而词本身代表了其 Value 向量。这种加权求和的方式使得模型能够根据当前上下文的需求，灵活地整合来自不同位置的信息，从而形成对当前词"it"的理解。这个简化的表达虽然忽略了很多细节，但基本表达了注意力机制中信息综合的核心思想。

3.注意力机制的本质:

Query 向量代表当前 token 的"问题"或"需求"。

Key 向量代表每个 token 可能提供的"信息"。

Value 向量是实际传递的"信息内容"。

在生成新 token 时，我们需要新的"问题"（Query）来查询所有历史"信息"（Key）并获取相关的"内容"（Value）。

注意力机制允许模型动态地"查阅"之前的信息。不同的信息源(早先的词)会根据其相关性获得不同程度的"注意力"。最终的表示是多个信息源的加权组合。

三. KV缓存：自回归模型的关键优化

1.为什么需要KV Cache:

1)计算效率问题:

在自回归生成过程中,模型需要为每个新token重新计算整个序列的Key和Value。这意味着,随着生成的token数量增加,计算量呈二次方增长。例如,在生成第20个token时,我们需要计算20次Key和Value;而在生成第21个token时,我们又需要重新计算21次。这种重复计算极大地降低了推理效率。

2)资源消耗:

反复计算不仅耗时,还会占用大量的计算资源,尤其是在处理长文本时。

3)延迟问题:

在实时对话或文本生成应用中,用户体验很大程度上取决于系统的响应速度。频繁的重复计算会导致明显的延迟。

2.KV Cache的原理:

KV Cache的核心思想是缓存并重用之前计算过的Key和Value,从而避免重复计算。

工作原理:

初始计算:

对于输入序列"A robot must obey",我们首先计算每个token的Key和Value。

缓存存储:

将计算得到的Key和Value存储在缓存中。例如:

- "A"的Key和Value存储在位置1
- "robot"的Key和Value存储在位置2
- "must"的Key和Value存储在位置3
- "obey"的Key和Value存储在位置4

生成新token:

当模型需要生成下一个token "the" 时:

a) 使用缓存中的Key和Value,无需重新计算。

b) 只需为新token "the" 计算Query。

c) 用这个新Query与缓存中的所有Key计算注意力分数。

d) 基于注意力分数和缓存的Value生成新的表示。

更新缓存:

生成 "the" 后,计算它的Key和Value,并将其添加到缓存的位置5。

重复过程:

对于后续的每个新token (如 "orders", "given", "it" 等),重复步骤3-4。

实例说明:

以生成 "it" (序列中的第9个token)为例:

缓存状态:

此时缓存中已有8个位置,存储了 "A robot must obey the orders given" 的Key和Value。

计算过程:

只需为 "it" 计算新的Query。

这个Query与缓存中的8个Key计算注意力分数。

使用这些分数和缓存的Value计算 "it" 的表示。

缓存更新:

计算 "it" 的Key和Value,添加到缓存的第9个位置。

3.KV Cache的优势:

1) 显著减少计算量:

对于长度为N的序列,传统方法需要O(N^2)的计算量,而使用KV Cache后仅需O(N)。

2)加速推理:

由于避免了重复计算,推理速度大幅提升。

3)内存权衡:

虽然需要额外的内存来存储缓存,但相比计算效率的提升,这是值得的权衡。

4) 适合长文本生成:

对于长文本生成任务,KV Cache的优势更加明显。

四、vLLM的核心技术：PagedAttention

在介绍vLLM的核心技术之前，我们首先需要理解传统LLM服务系统面临的主要挑战。这些挑战正是vLLM试图解决的问题，也是理解PagedAttention重要性的关键。

1. 传统LLM服务的效率瓶颈

传统的LLM服务系统在处理大规模语言模型时面临以下几个主要问题:

a) 内存碎片化

- 内部碎片：系统往往为每个请求预分配最大可能长度的连续内存，导致大量内存被浪费。

- 外部碎片：不同长度的请求导致内存中出现难以利用的小块空闲空间。

b) 内存利用率低

由于预分配和碎片化问题，大量GPU内存被浪费，限制了系统能够同时处理的请求数量。

c) 缺乏灵活性

将整个KV缓存存储在连续内存空间中，难以适应动态变化的工作负载。

d) 内存共享困难

在并行采样或束搜索等场景中，难以实现高效的内存共享。

e) 计算冗余

无法有效共享和重用计算结果，导致大量重复计算。

f) 批处理效率低下

静态批处理策略难以在延迟和吞吐量之间取得良好平衡。

g) 调度不灵活

粗粒度的调度策略难以根据实时工作负载动态调整资源分配。

h) 扩展性限制

在模型并行和数据并行方面的支持不够优化，限制了系统的扩展性。

i) 长序列处理效率低

随着序列长度增加，性能急剧下降。

j) 动态长度处理能力差

难以高效处理动态变化的输入和输出序列长度。

2. PagedAttention: vLLM的核心创新

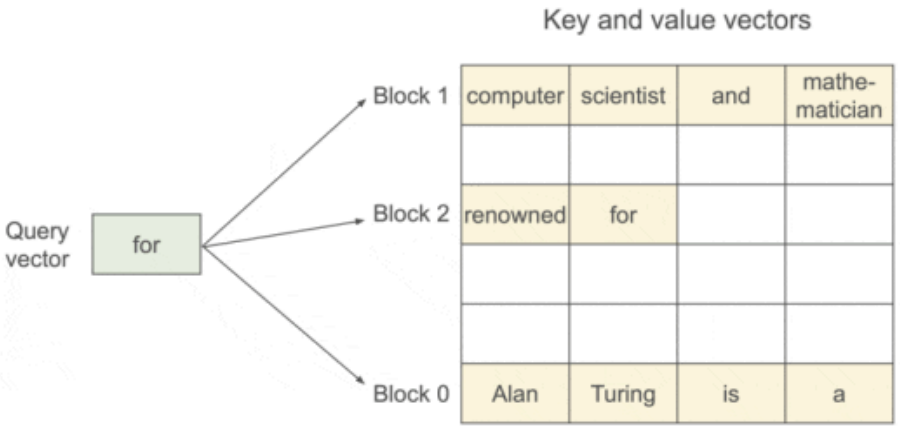
加州大学伯克利分校的研究团队开发了vLLM，一个开源的LLM推理和服务库。vLLM的核心创新在于其独特的注意力算法——PagedAttention，它彻底改变了LLM服务的效率和性能。

PagedAttention的灵感来源于操作系统中的虚拟内存和分页概念。传统的注意力算法要求将键(key)和值(value)张量连续存储在内存中，而PagedAttention允许将它们存储在非连续的内存空间中。简而言之，PagedAttention 背后的想法是创建映射到 GPU 内存中的物理块的连续虚拟块。每个块都旨在存储预定义数量的标记的键值对张量。所有块都是虚拟连续的，并映射到碎片化 GPU 内存中在推理期间按需分配的物理非连续块。内存中还会创建一个简单的索引表，以将虚拟块与物理块关联起来。PagedAttention 的内核会根据需要获取这些块。由于块的大小有限，系统会获取较少数量的键值张量，因此这种方法非常高效。

PagedAttention的工作原理可以通过以下几个关键方面来理解：

1. 非连续内存访问和查询机制

PagedAttention允许从非连续的内存位置高效地检索和使用相关信息。



1). Query vector（查询向量）：

图中左侧绿色方框中的"for"是当前的查询向量。在注意力机制中，查询向量用于与key vectors进行比较，以确定应该关注哪些信息。

2). Key and value vectors（键值向量）：

右侧的表格代表了存储在内存中的key和value向量。这些向量被分成了多个块（Block 0, Block 1, Block 2）。

3). 内存块（Blocks）：

- Block 0 包含 "Alan Turing is a"
- Block 1 包含 "computer scientist and mathematician"
- Block 2 包含 "renowned for"

4). 非连续内存访问：

图中的箭头显示了查询向量"for"如何同时访问这三个非连续的内存块。这正是PagedAttention的核心创新——它允许从非连续的内存位置高效地检索和使用相关信息。

5). 灵活的内存管理：

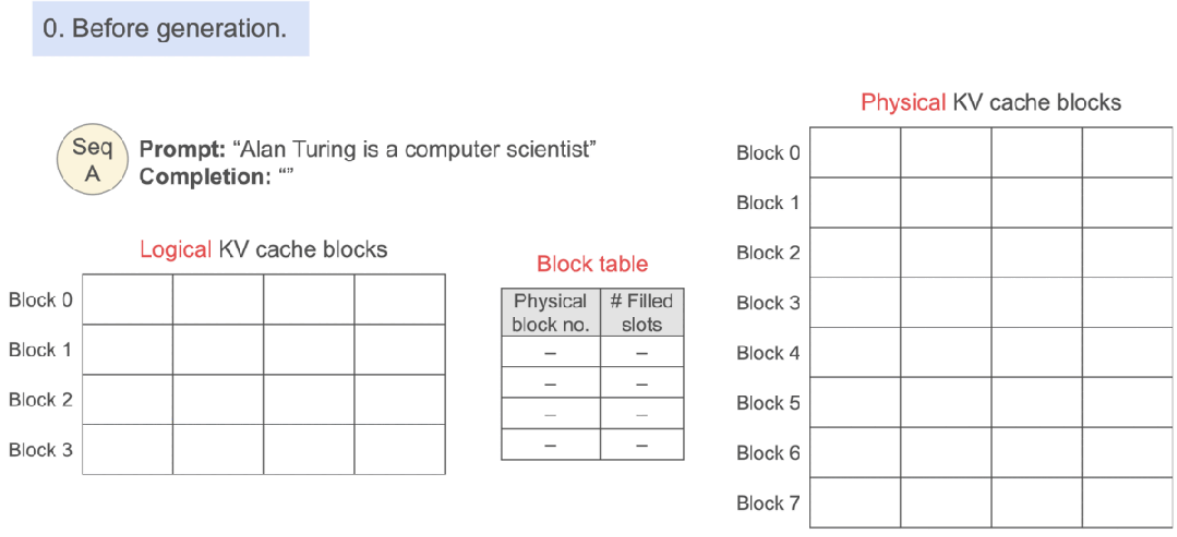
这种设计使得系统可以更灵活地管理内存，不需要所有相关的key和value向量都连续存储在一起。

6). 高效计算：

尽管内存块是非连续的，PagedAttention算法仍然能够高效地计算注意力分数，将查询向量与所有相关的key向量进行比较。

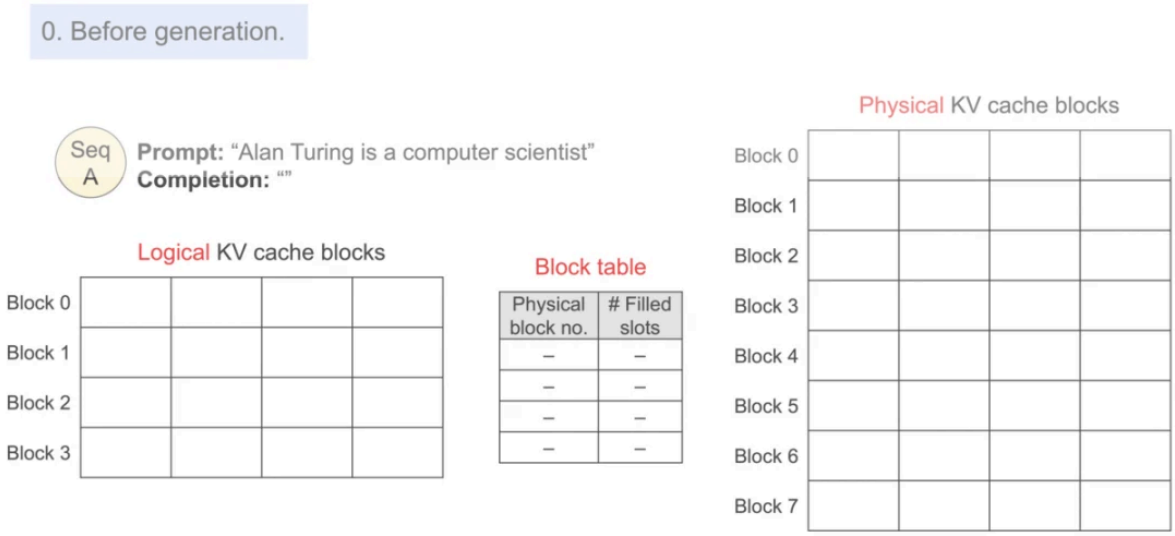
这种方法的优势在于它可以更有效地利用GPU内存，减少内存碎片，并允许更灵活的内存分配和管理策略。这对于处理变长序列和批量处理多个请求特别有利，从而显著提高了大型语言模型的服务效率和吞吐量。

2. 动态内存管理和生成过程



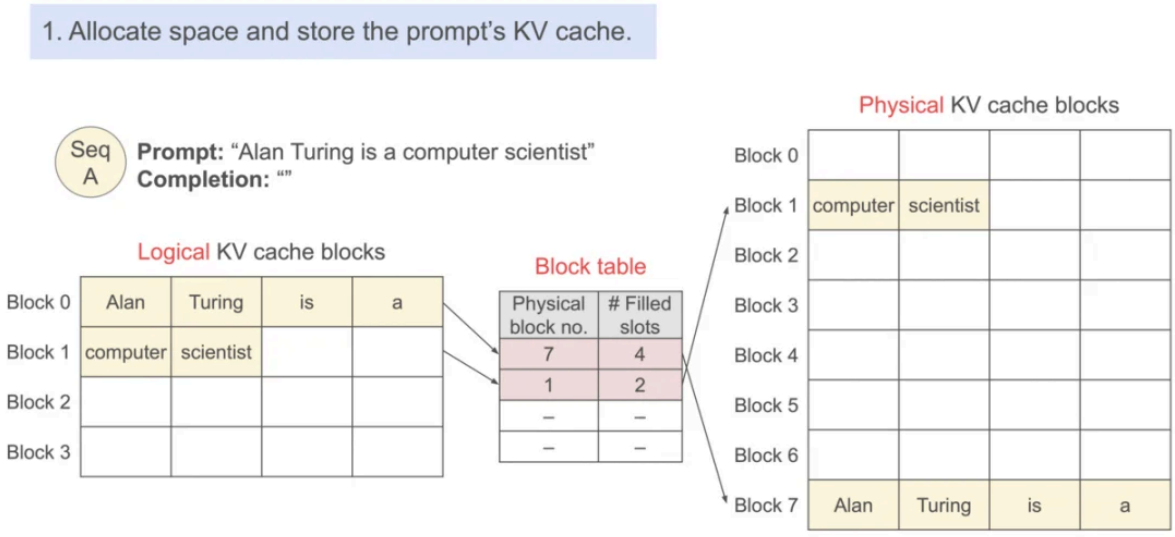
1). 初始状态：

- 提示词是"Alan Turing is a computer scientist"
- 逻辑KV缓存块、物理KV缓存块和块表都是空的



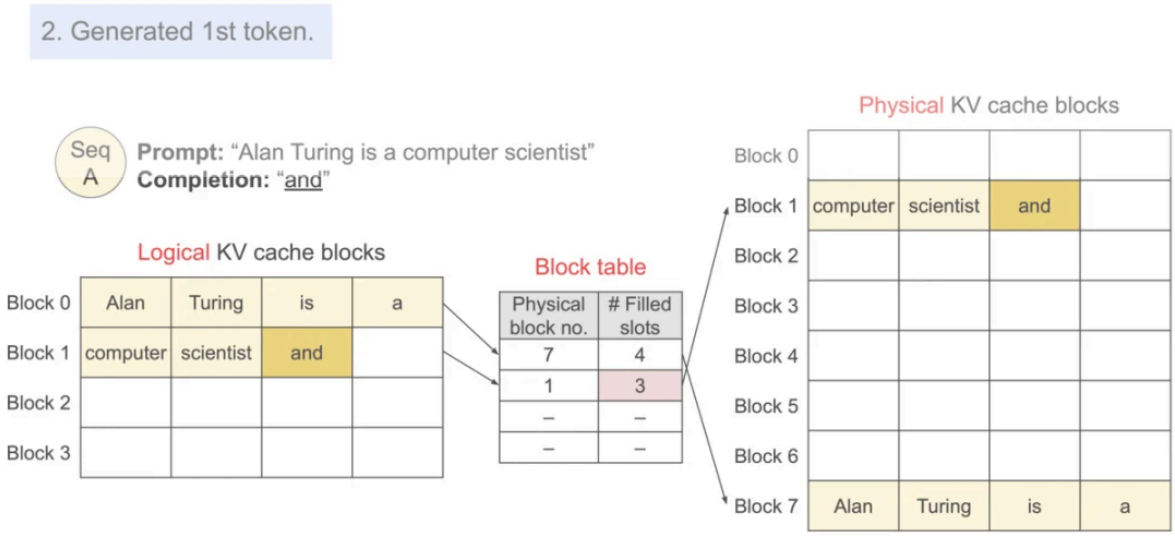
2). 分配空间并存储提示词的KV缓存:

- 逻辑块0和1被填充了提示词的tokens
- 物理块7和1被分配并填充相应的数据
- 块表记录了逻辑块到物理块的映射关系



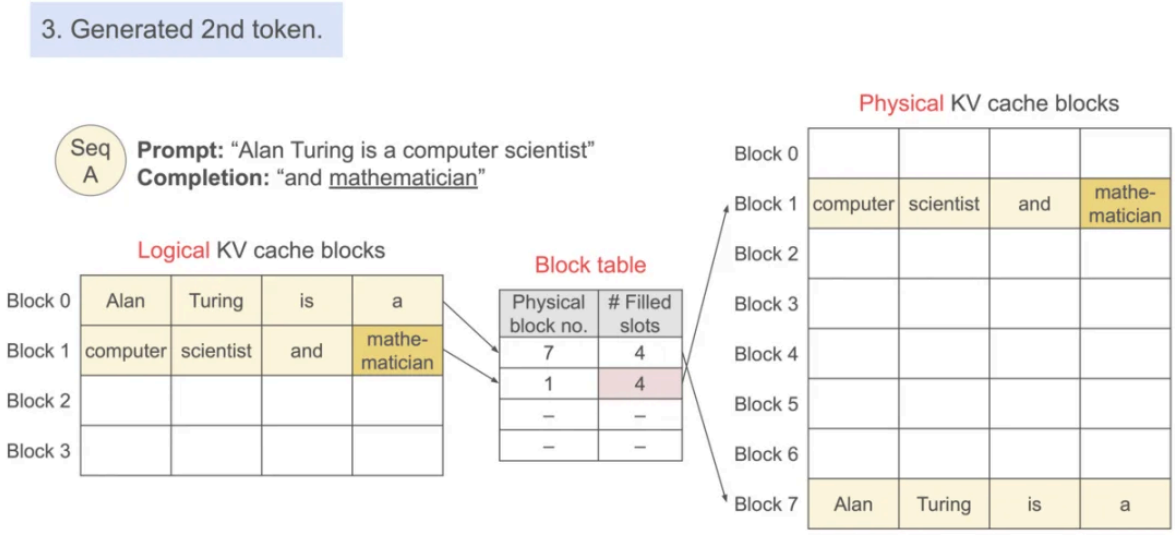
3). 生成第一个token "and":

- "and"被添加到逻辑块1
- 物理块1中相应位置也被填充
- 块表中填充槽数量增加



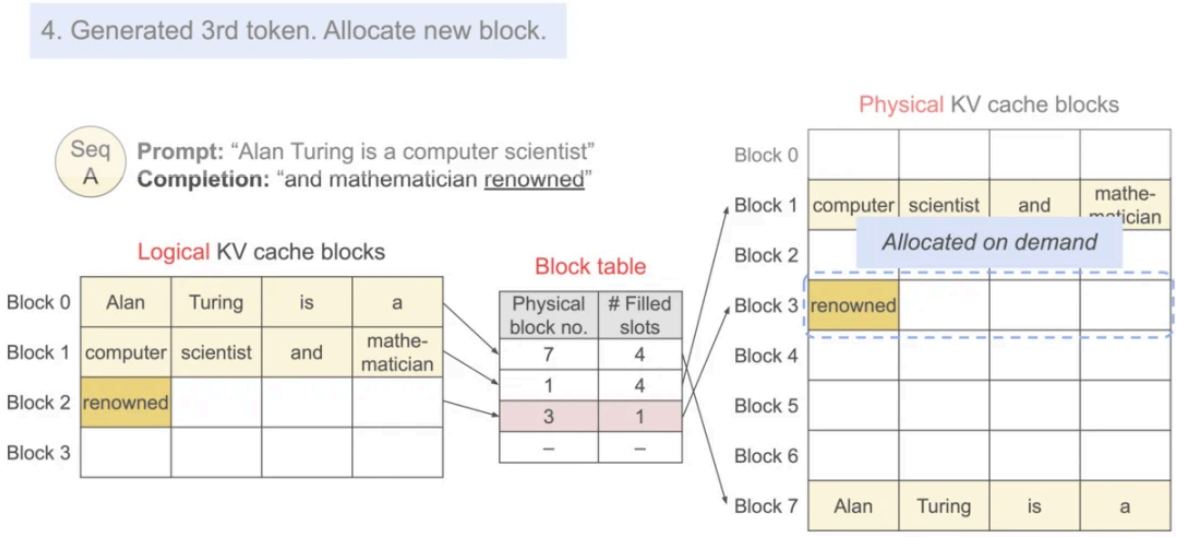
4). 生成第二个token "mathematician":

- "mathematician"被添加到逻辑块1
- 物理块1被完全填满



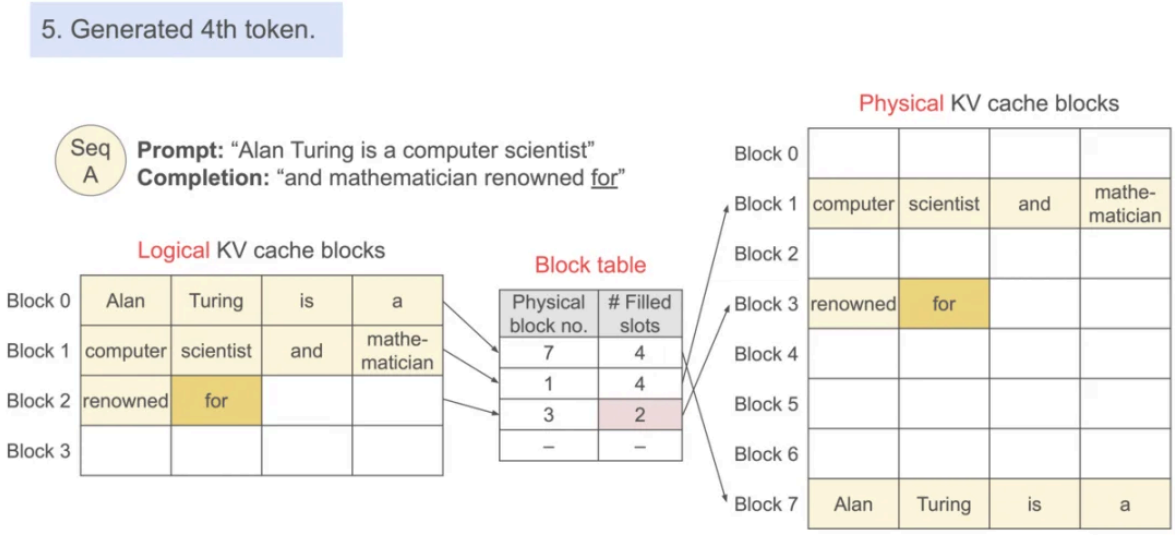
5). 生成第三个token "renowned":

- 由于逻辑块1已满,新的逻辑块2被创建
- 物理块3被按需分配
- "renowned"被添加到新的逻辑和物理块中



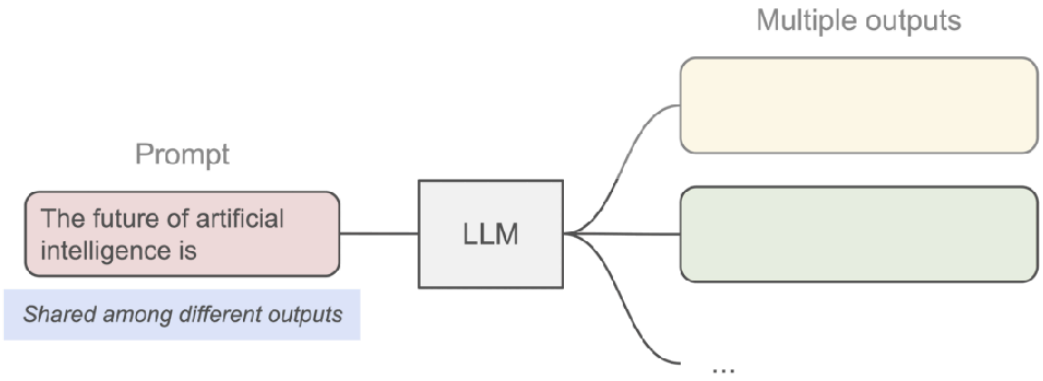
6). 生成第四个token "for":

- "for"被添加到逻辑块2
- 物理块3中相应位置也被填充



这个过程展示了PagedAttention如何灵活地管理内存,通过将逻辑块映射到非连续的物理块,实现了高效的内存利用和动态扩展。

3. 内存共享



1). 输入prompt：

"The future of artificial intelligence is" 作为输入提示。

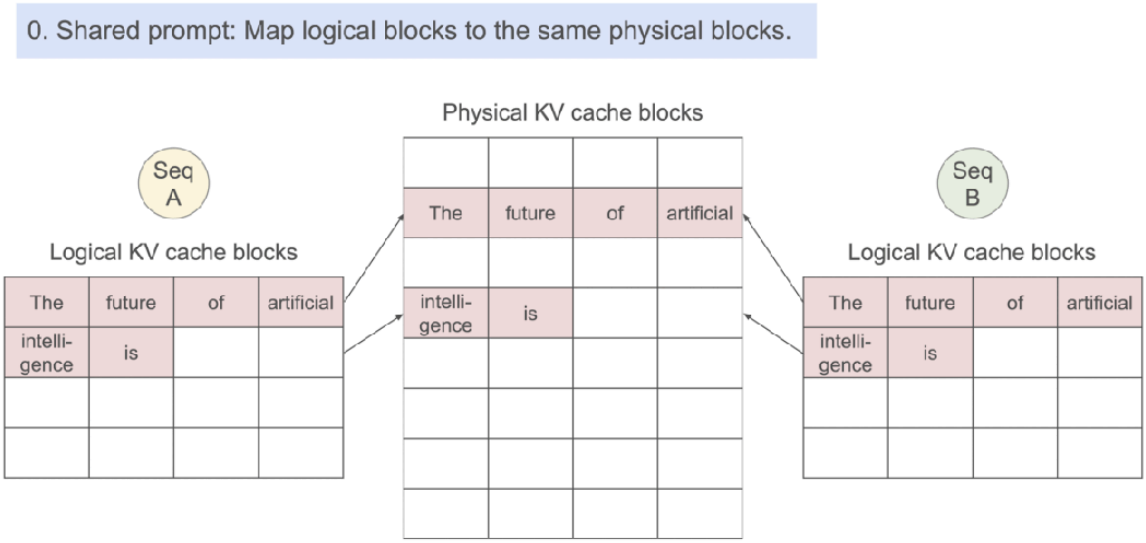
2). 共享prompt处理：

- LLM（大语言模型）首先处理这个prompt。
- prompt的处理结果（key和value向量）被存储在KV缓存中。
- 这部分内存是共享的，意味着所有后续的并行输出都可以使用这些相同的计算结果。

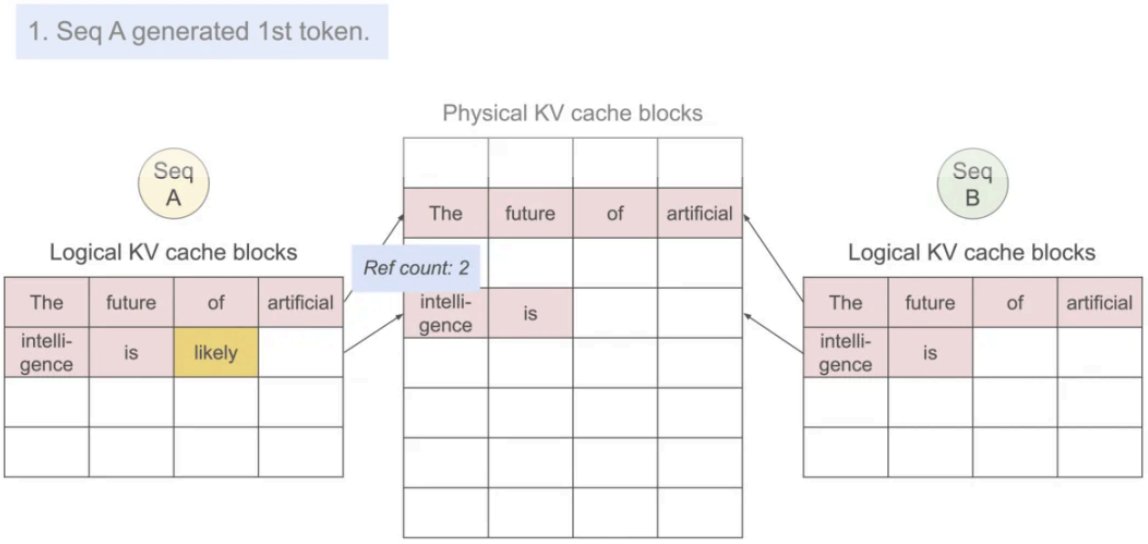
3). 多个输出：

- LLM基于相同的prompt生成多个不同的输出。
- 每个输出代表一个不同的续写或完成方案。

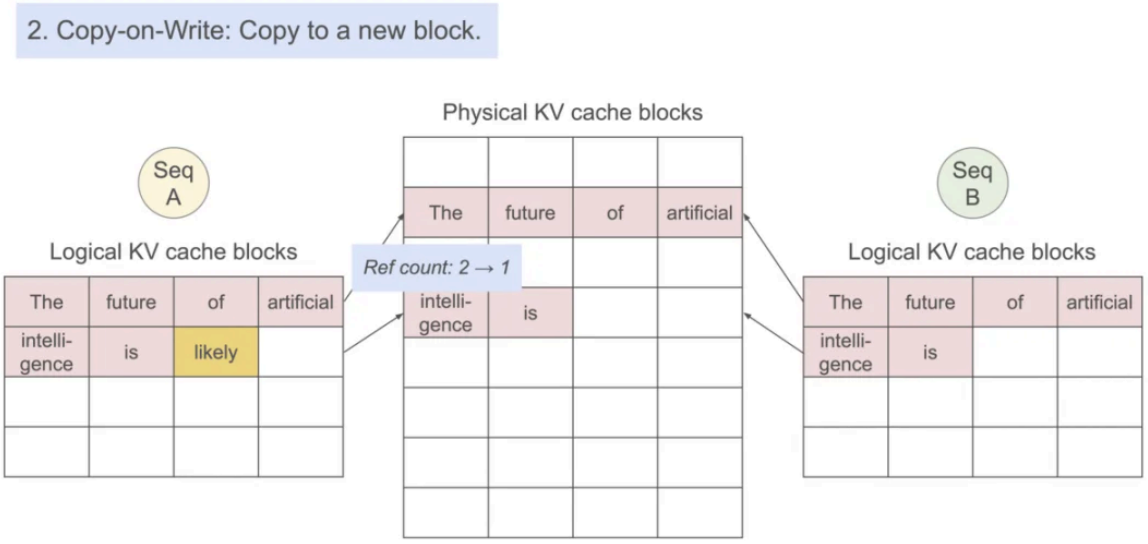
4. 内存共享实现机制演示



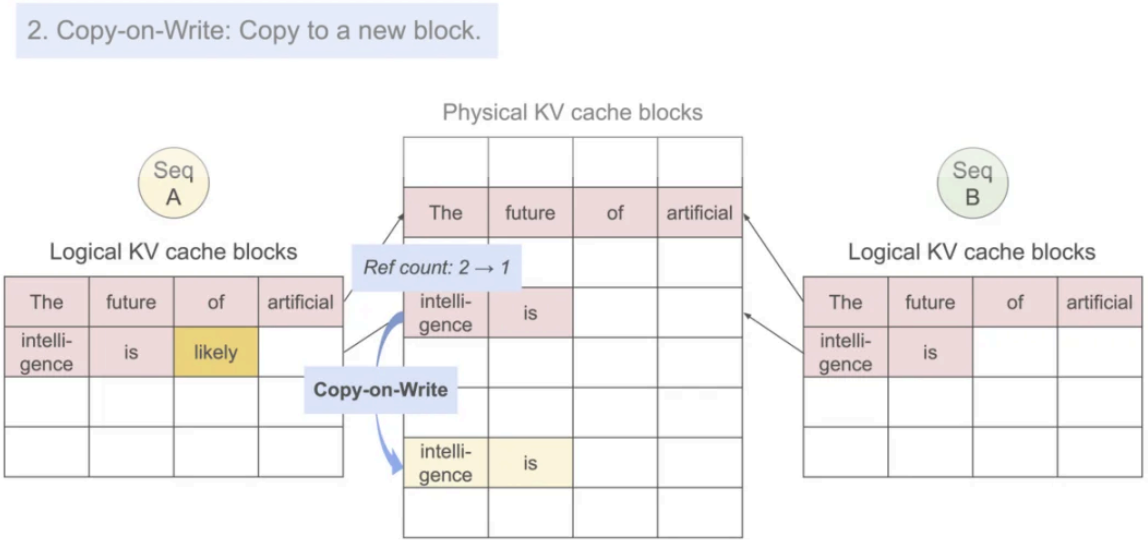
1).初始状态，序列A和B共享相同的初始提示："The future of artificial intelligence is"。物理KV缓存块存储了这个共享提示，两个序列的逻辑KV缓存块都指向相同的物理块，引用计数为2。



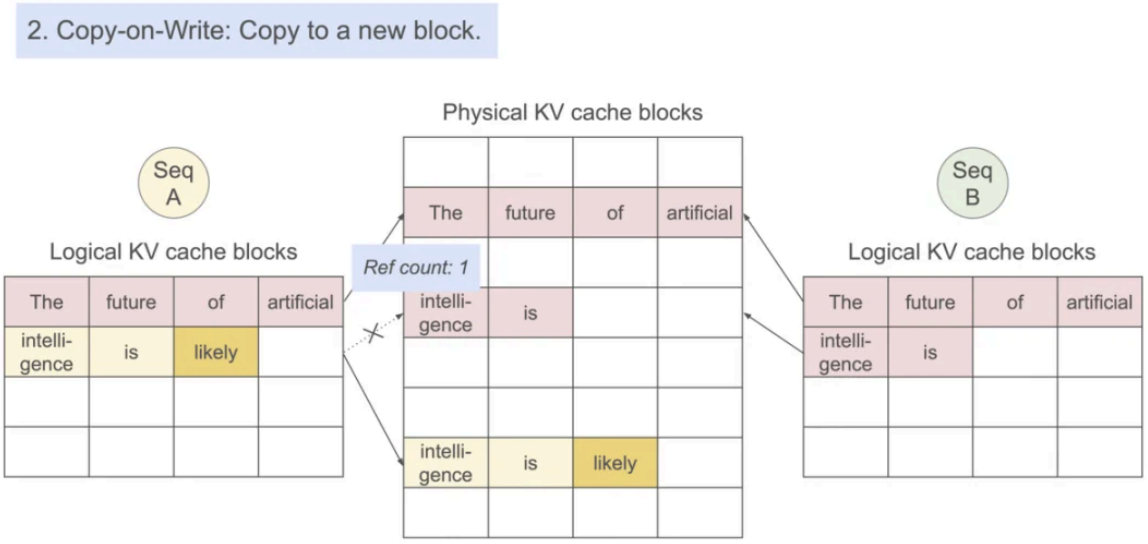
2).序列A生成了第一个新token "likely"。由于这是一个写操作，触发了Copy-on-Write机制。系统创建了一个新的物理块来存储"intelligence is likely"。序列A的逻辑块现在指向这个新的物理块。



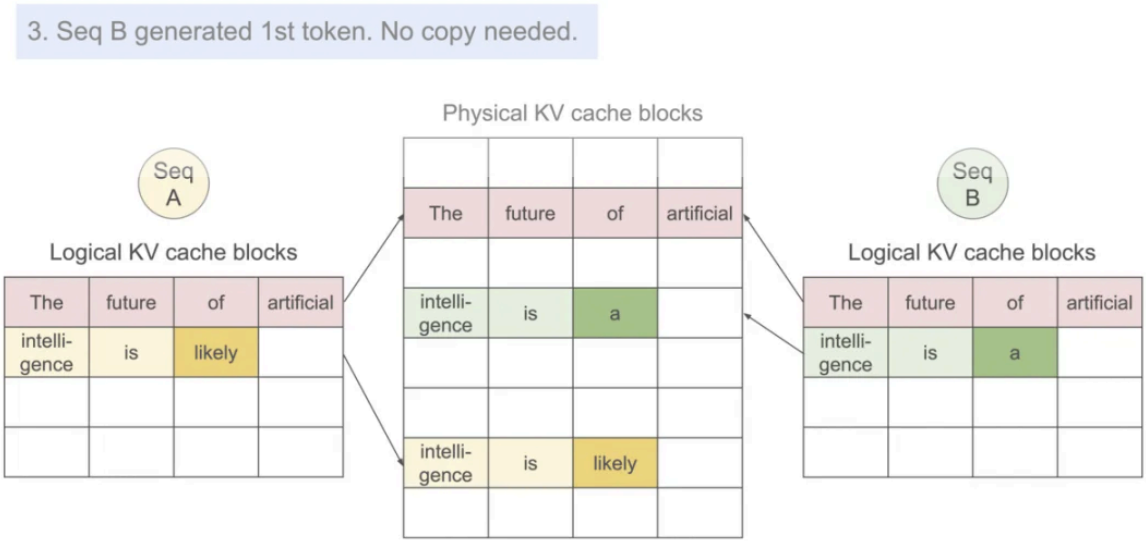
3).Copy-on-Write过程的细节。新的物理块被创建，包含"intelligence is"和新生成的"likely"。序列A的逻辑块更新以指向这个新块。原始物理块的引用计数从2减少到1。



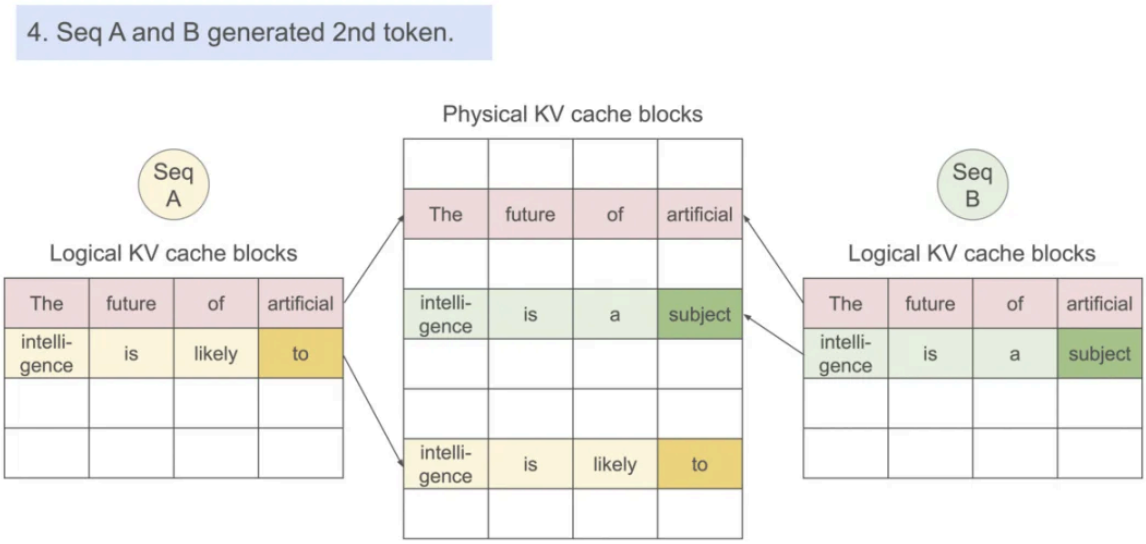
4).图3Copy-on-Write过程完成。序列A的逻辑块完全指向新的物理块，包含"intelligence is likely"。原始物理块仍然被序列B引用。



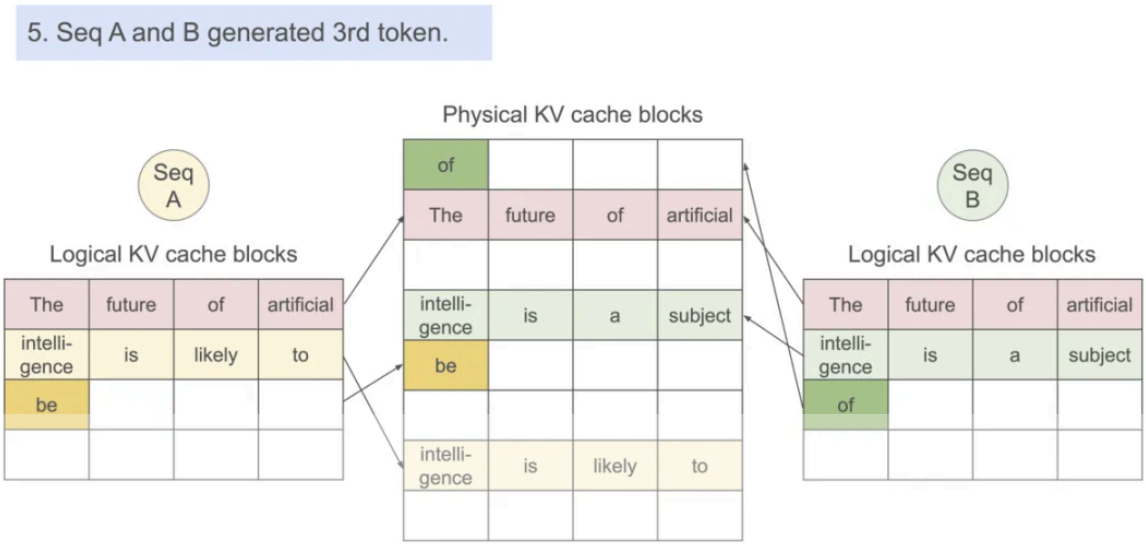
5).序列B生成了它的第一个新token "a"。由于这个token与序列A不同，不需要复制共享的物理块。序列B在自己的逻辑块中添加了新token "a"。



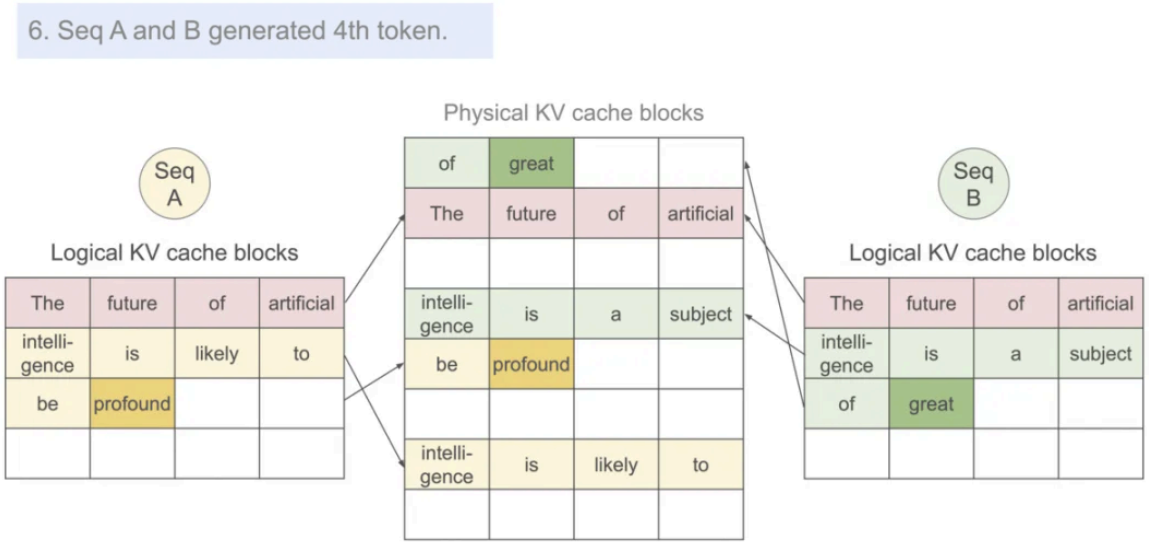
6).两个序列继续生成新的token。序列A生成 "to"，序列B生成 "subject"。每个序列在自己的逻辑块中继续添加新的token。



7).序列继续生成。序列A生成 "be"，序列B生成 "of"。新的物理块被创建来存储这些不同的token。



8).最后一步。序列A生成 "profound"，序列B生成 "great"。每个序列继续在各自的逻辑块中添加新token。



这个过程展示了PagedAttention如何高效地管理内存，通过共享初始内容和仅在必要时创建新的物理块来最大化内存利用率，同时允许不同序列独立生成内容。

四、vLLM其它优化技术

- 连续批处理：传入的请求被连续批处理在一起，以最大限度地提高硬件利用率并减少计算浪费，最大限度地减少空闲时间。
- 量化：vLLM 利用 FP16 等量化技术，通过以较低的精度表示 KV 缓存来优化内存使用，从而减少内存占用并加快计算速度。
- 优化的 CUDA 内核：vLLM 手动调整在 GPU 上执行的代码以实现最佳性能。例如，对于融合重塑和块写入，开发了优化的内核，将新的 KV 缓存拆分为块，重塑它们以实现高效的内存访问，并根据块表保存它们，所有这些都融合到单个内核中以减少开销。

五、总结

vLLM 解决了 LLM 部署中的一个关键瓶颈：推理和服务效率低下。使用创新的 PagedAttention 技术，vLLM 优化了核心注意操作期间的内存使用，从而显著提高了性能。这意味着更快的推理速度和在资源受限的硬件上运行 LLM 的能力。

参考及附图来源：

1.https://jalammar.github.io/illustrated-gpt2/

2.https://blog.vllm.ai/2023/06/20/vllm.html